

Optimization, in plain English

Companion notes for Lecture 10 • ZC416 Mathematical Foundations for ML / DS

BITS Pilani WILP • Read this beside the slides. Every number in here is worked out in full.

The whole lecture in one story

You are standing somewhere on a hillside in thick fog. You cannot see the valley. All you can feel is the slope under your feet. So you take a small step downhill, feel again, step again. That is **gradient descent**, and that is essentially how every model in this course learns.

Four things can go wrong on that hillside, and this entire lecture is about those four things:

1. **You get dropped in a bad place.** Where you start matters. *(Initialisation)*
2. **You reach the bottom of the *wrong* valley**, or you cross a huge flat plateau where the ground feels level and you crawl. *(Local optima and flat regions)*
3. **The valley is not a bowl, it is a canyon.** Steep across, gentle along. You bounce off the walls and barely move forward. *(Differential curvature)*
4. **You reach the bottom, and the bottom is a lie.** You have fitted the training data perfectly and learnt nothing true. *(Overfitting)*

Everything else in the slides — mean-centering, normalisation, contour plots, second derivatives — is machinery for handling those four problems.

Minimising the loss is *not* the goal. Landing in a place that also behaves well on data you have never seen is the goal. Gradient descent is just the vehicle, and it has a few well-known handling problems.

Contents

The whole lecture in one story	1
1 The loss function: a scorecard for wrongness	2
2 Where do you start? (Initialisation)	3
3 Overfitting: getting full marks by memorising	4
4 Feature processing: why units wreck learning	6
5 Local optima and flat regions	7
6 Differential curvature: the canyon problem	10
7 Revisiting normalisation: the fix, with the numbers	12
8 The whole toolkit on one page	15

9 Check yourself 15

Glossary in ordinary words 16

1. The loss function: a scorecard for wrongness

A model makes a guess. The truth is known for the training data. The loss function is simply the bill for being wrong.

The slides use the least-squares loss

$$J(\mathbf{w}) = \sum_{i=1}^n \|\mathbf{w}^T \mathbf{X}_i - y_i\|^2, \quad \hat{y}_i = \mathbf{w}^T \mathbf{X}_i.$$

Read it right to left, in words:

- $\mathbf{X}_i$ is *one* training example — one row of your table, one customer, one patient, one house.
- $\mathbf{w}^T \mathbf{X}_i$ is the model's prediction: multiply each feature by its importance and add up.
- $\mathbf{w}^T \mathbf{X}_i - y_i$ is the mistake on that one row.
- Square it, so that being -5 off is punished exactly as much as being $+5$ off, and so that big mistakes hurt disproportionately more than small ones.
- Sum over all n rows: total wrongness.

In plain words

$J(\mathbf{w})$ is a machine that eats a set of weights and spits out a single number: “how bad”. Training means turning the weight knobs until that number is small. Nothing more mystical than that.

Worked example 1: the smallest possible training run

Three data points, one feature, no intercept. Model: $\hat{y} = wx$.

x	y
1	2
2	5
3	7

Step 1 — write the scorecard.

$$J(w) = (1w - 2)^2 + (2w - 5)^2 + (3w - 7)^2.$$

Step 2 — expand each square.

$$\begin{aligned} (w - 2)^2 &= w^2 - 4w + 4 \\ (2w - 5)^2 &= 4w^2 - 20w + 25 \\ (3w - 7)^2 &= 9w^2 - 42w + 49 \end{aligned}$$

Step 3 — add them up.

$$J(w) = (1 + 4 + 9)w^2 - (4 + 20 + 42)w + (4 + 25 + 49) = 14w^2 - 66w + 78.$$

Step 4 — differentiate.

$$\frac{dJ}{dw} = 28w - 66.$$

Step 5 — set the slope to zero.

$$28w - 66 = 0 \implies w^* = \frac{66}{28} = \frac{33}{14} \approx 2.357.$$

Step 6 — check it is a minimum. $\frac{d^2J}{dw^2} = 28 > 0$. Curving upwards, so yes, a minimum. The lowest possible bill is

$$J(w^*) = 14 \left(\frac{33}{14}\right)^2 - 66 \cdot \frac{33}{14} + 78 = \frac{1089}{14} - \frac{2178}{14} + \frac{1092}{14} = \frac{3}{14} \approx 0.214.$$

Step 7 — now do it the fog way (gradient descent), $w_0 = 0$, step size $\eta = 0.02$:

$$w \leftarrow w - \eta (28w - 66)$$

step	w	slope $28w - 66$	$J(w)$
0	0.000	-66.00	78.00
1	1.320	-29.04	15.19
2	1.901	-12.78	2.99
3	2.156	-5.62	0.79
4	2.269	-2.47	0.32
5	2.318	-1.09	0.24
∞	2.357	0	0.214

Same answer, reached by feel instead of by formula. For a one-parameter problem the formula wins. For a neural network with 70 billion parameters there is no formula, only the fog.

2. Where do you start? (Initialisation)

Gradient descent needs a first guess w_0 . The slides say: usually pick each component randomly from $[-1, +1]$, or from $[0, 1]$ if the parameters must stay non-negative.

Why such small, boring numbers?

- **Because arithmetic breaks.** Suppose a feature has value $x = 200$ and you initialise $w = 5000$. Then $\hat{y} = 10^6$ while the truth is maybe 7. The squared error is about 10^{12} , the gradient is about 10^9 , and the very first update throws w to some absurd value. Two more steps and you have inf or NaN . This is the overflow the slide warns about. Underflow is the mirror image: start with $w = 10^{-12}$ inside a product of many such terms and everything rounds to zero, so no learning signal survives.
- **Because the parameters talk to each other.** If two weights are correlated — moving one is only sensible together with the other — then a careless start can drop you on the far side of a ridge, and you will descend into the wrong valley (Section 5).
- **Because symmetry is fatal in networks.** Initialise every weight in a layer to the same value and every neuron receives the same gradient forever; they stay identical clones and the layer behaves like a single neuron. Randomness is what breaks the tie.

In plain words

Starting values are a parachute drop. Drop yourself somewhere small, random and central; do not drop yourself off the edge of the map, and do not drop the whole team on the exact same square metre.

3. Overfitting: getting full marks by memorising

The situation

Four inputs x_1, x_2, x_3, x_4 , one output y , and a model with five knobs:

$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + w_5.$$

The slide's key observation: with *fewer than five* training rows you are solving fewer equations than you have unknowns. Such a system has infinitely many exact solutions, so you can always drive the training error to *exactly zero*. That sounds like a triumph. It is usually a disaster.

In plain words

Five unknowns, three equations. It is like being told “two numbers add to 10” and being asked what they are: (5, 5), (0, 10), (−3, 13) all work. The data has not pinned the answer down; your optimiser will just hand you *one* of the infinitely many, and it has no idea which one is meaningful.

The slide's data

x_1	x_2	x_3	x_4	y
61	2	3	0.1	59
40	0	4	0.5	40
68	0	10	1.0	70

We are told the real-world truth is $y \approx x_1$. Look at the column: $61 \rightarrow 59, 40 \rightarrow 40, 68 \rightarrow 70$. Yes — y is basically x_1 with a little noise on top.

Candidate A: the honest model

Take $w_1 = 1$ and $w_2 = w_3 = w_4 = w_5 = 0$, i.e. $\hat{y} = x_1$.

row 1:	$\hat{y} = 61,$	$y = 59,$	error = +2,	squared = 4
row 2:	$\hat{y} = 40,$	$y = 40,$	error = 0,	squared = 0
row 3:	$\hat{y} = 68,$	$y = 70,$	error = −2,	squared = 4

$$J = 4 + 0 + 4 = 8.$$

Not zero. Close to zero, but not zero.

Candidate B: the memoriser

Now ignore x_1 completely ($w_1 = 0, w_4 = 0$) and try to hit all three rows exactly using only x_2, x_3 and the constant w_5 . That is three equations in three unknowns:

$$\begin{aligned} 2w_2 + 3w_3 + w_5 &= 59 && \text{(row 1)} \\ 4w_3 + w_5 &= 40 && \text{(row 2)} \\ 10w_3 + w_5 &= 70 && \text{(row 3)} \end{aligned}$$

Step 1. Subtract (row 2) from (row 3) to kill w_5 :

$$10w_3 - 4w_3 = 70 - 40 \implies 6w_3 = 30 \implies \boxed{w_3 = 5}.$$

Step 2. Put $w_3 = 5$ into (row 2):

$$4(5) + w_5 = 40 \implies 20 + w_5 = 40 \implies \boxed{w_5 = 20}.$$

Step 3. Put both into (row 1):

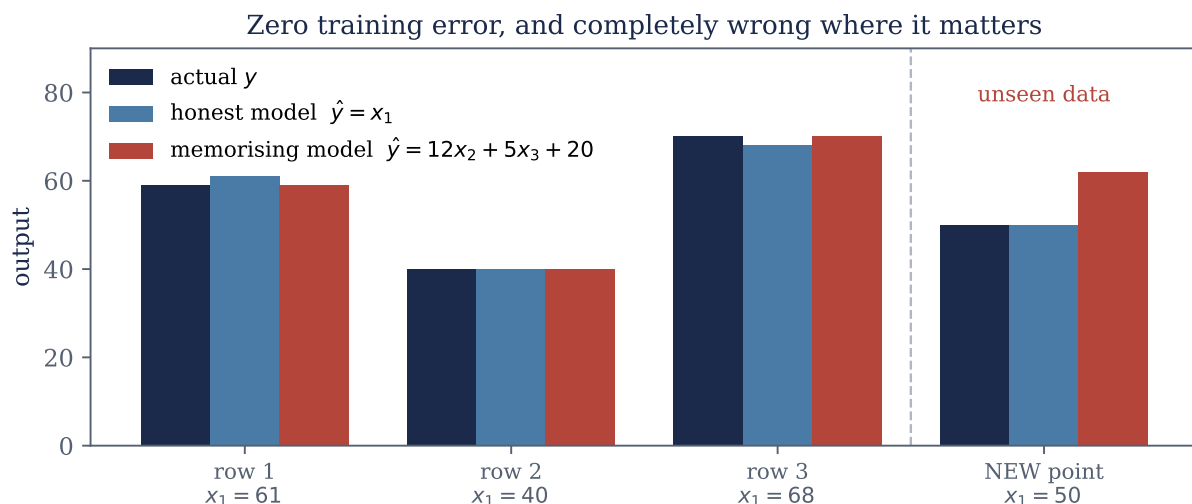
$$2w_2 + 3(5) + 20 = 59 \implies 2w_2 + 35 = 59 \implies 2w_2 = 24 \implies \boxed{w_2 = 12}.$$

So the memorising model is

$$\hat{y} = 12x_2 + 5x_3 + 20, \quad J = 0 \text{ exactly.}$$

Slide 8 quotes this solution as $w_2 = 7$, $w_3 = 5$, $w_5 = 20$. Check row 1 with those numbers: $7(2) + 5(3) + 20 = 14 + 15 + 20 = 49$, but $y = 59$ — so that particular triple does *not* give zero training error. Solving the three equations honestly, as above, gives $w_2 = 12$. The lesson of the slide is completely unaffected; only the arithmetic needed a nudge. Use $w_2 = 12$ if you reproduce this example in an assignment.

Which model would you rather ship?



On the three training rows, Candidate B is *perfect* ($J = 0$) and Candidate A is merely very good ($J = 8$). Any optimiser told “minimise J ” will happily choose B.

Now show both models one new row that they have never seen — a real-world row with $x_1 = 50$, $x_2 = 1$, $x_3 = 6$, $x_4 = 0.3$, and true $y \approx 50$:

Candidate A: $\hat{y} = 50$

error ≈ 0

Candidate B: $\hat{y} = 12(1) + 5(6) + 20 = 62$

error ≈ 12

Candidate B was not learning; it was memorising three rows using variables that have nothing to do with the answer. It is the student who scores 100% because they memorised last year’s paper, and 20% on this year’s.

More knobs than facts $\Rightarrow$ zero training error is available $\Rightarrow$ zero training error means nothing. Judge a model on data it has not seen.

In plain words

This is also the intuition behind *regularisation*, which you will meet later: add a penalty like $\lambda \sum_j w_j^2$ to the loss. Candidate B has weights 12 and 5 and 20; Candidate A has a single weight of 1. The penalty makes B expensive and A cheap, so the optimiser stops preferring the memoriser.

4. Feature processing: why units wreck learning

The problem, stated with the slides' example

Wealth y is modelled from age $x_1 \in [0, 100]$ and years of college $x_2 \in [0, 10]$:

$$y = w_1 x_1^2 + w_2 x_2^2.$$

The sensitivity of the output to each knob is

$$\frac{\partial y}{\partial w_1} = x_1^2, \quad \frac{\partial y}{\partial w_2} = x_2^2.$$

Put in typical values, say $x_1 = 50$ and $x_2 = 4$:

$$\frac{\partial y}{\partial w_1} = 2500, \quad \frac{\partial y}{\partial w_2} = 16.$$

The same nudge to w_1 changes the answer roughly **156** times more than the same nudge to w_2 .

In plain words

Two knobs on the same machine. One is a hair-trigger; a millimetre of turn swings the output wildly. The other is stiff; you can spin it and almost nothing happens. Gradient descent uses *one* learning rate η for both knobs. Any η that is gentle enough not to over-turn the hair-trigger is far too gentle to move the stiff one. You bounce in one direction and crawl in the other.

Slide 9 ends with "... we take small steps in respect of w_2 and large steps in respect of w_1 " and slide 10 continues from there. Note the mechanism carefully: the *step size* η is the same for both; it is the *gradient* that is huge for w_1 and tiny for w_2 , so the *actual movement* $\eta \times \text{gradient}$ is large in w_1 and small in w_2 . Large movements in w_1 overshoot and oscillate; small movements in w_2 creep. That combination is what makes convergence painfully slow.

The three standard repairs

Technique	Formula	What it does, in words
Mean-centering	$x_{ij} \leftarrow x_{ij} - \mu_j$	Slides the cloud of data so its centre sits at the origin. Removes the “everything is around 15” offset that makes all columns look alike.
Standardisation (feature normalisation)	$x_{ij} \leftarrow \frac{x_{ij} - \mu_j}{\sigma_j}$	Also rescales so every column has spread 1. After this, “one unit” means the same amount of <i>surprise</i> in every column.
Min-max normalisation	$x_{ij} \leftarrow \frac{x_{ij} - \min_j}{\max_j - \min_j}$	Squeezes every column into $[0, 1]$. Simple and bounded, but a single crazy outlier squashes everyone else into a corner.

Normalisation does not change what the data *means*. It changes the *units* so that one learning rate can serve all the knobs at once.

5. Local optima and flat regions

The slides' function, worked out completely

$$F(x) = (x - 1)^2((x - 3)^2 - 1).$$

Step 1 — differentiate with the product rule. With $u = (x - 1)^2$ and $v = (x - 3)^2 - 1$:

$$u' = 2(x - 1), \quad v' = 2(x - 3),$$

$$F'(x) = u'v + uv' = 2(x - 1)((x - 3)^2 - 1) + (x - 1)^2 \cdot 2(x - 3).$$

Step 2 — pull out the common factor $2(x - 1)$.

$$F'(x) = 2(x - 1) \left[(x - 3)^2 - 1 + (x - 1)(x - 3) \right].$$

Step 3 — simplify the bracket.

$$\begin{aligned} (x - 3)^2 - 1 &= x^2 - 6x + 9 - 1 = x^2 - 6x + 8 \\ (x - 1)(x - 3) &= x^2 - 4x + 3 \\ \text{sum} &= 2x^2 - 10x + 11 \end{aligned}$$

So

$$F'(x) = 2(x - 1)(2x^2 - 10x + 11)$$

Step 4 — set $F'(x) = 0$. Either $x - 1 = 0$, giving $x = 1$; or $2x^2 - 10x + 11 = 0$, giving

$$x = \frac{10 \pm \sqrt{100 - 4 \cdot 2 \cdot 11}}{2 \cdot 2} = \frac{10 \pm \sqrt{12}}{4} = \frac{10 \pm 2\sqrt{3}}{4} = \frac{5 \pm \sqrt{3}}{2}.$$

Numerically, $\frac{5 - \sqrt{3}}{2} = 1.634$ and $\frac{5 + \sqrt{3}}{2} = 3.366$. Three stationary points: $x = 1, 1.634, 3.366$.

Step 5 — which is which? Expand F' and differentiate again.

$$\begin{aligned} F'(x) &= 2(x-1)(2x^2 - 10x + 11) \\ &= 2(2x^3 - 10x^2 + 11x - 2x^2 + 10x - 11) \\ &= 2(2x^3 - 12x^2 + 21x - 11) \\ &= 4x^3 - 24x^2 + 42x - 22 \end{aligned}$$

$$F''(x) = 12x^2 - 48x + 42$$

x	$F''(x)$	sign	verdict
1	$12 - 48 + 42 = 6$	> 0	bowl-shaped $\Rightarrow$ minimum
1.634	$32.04 - 78.43 + 42 = -4.39$	< 0	dome-shaped $\Rightarrow$ maximum
3.366	$135.96 - 161.57 + 42 = 16.39$	> 0	bowl-shaped $\Rightarrow$ minimum

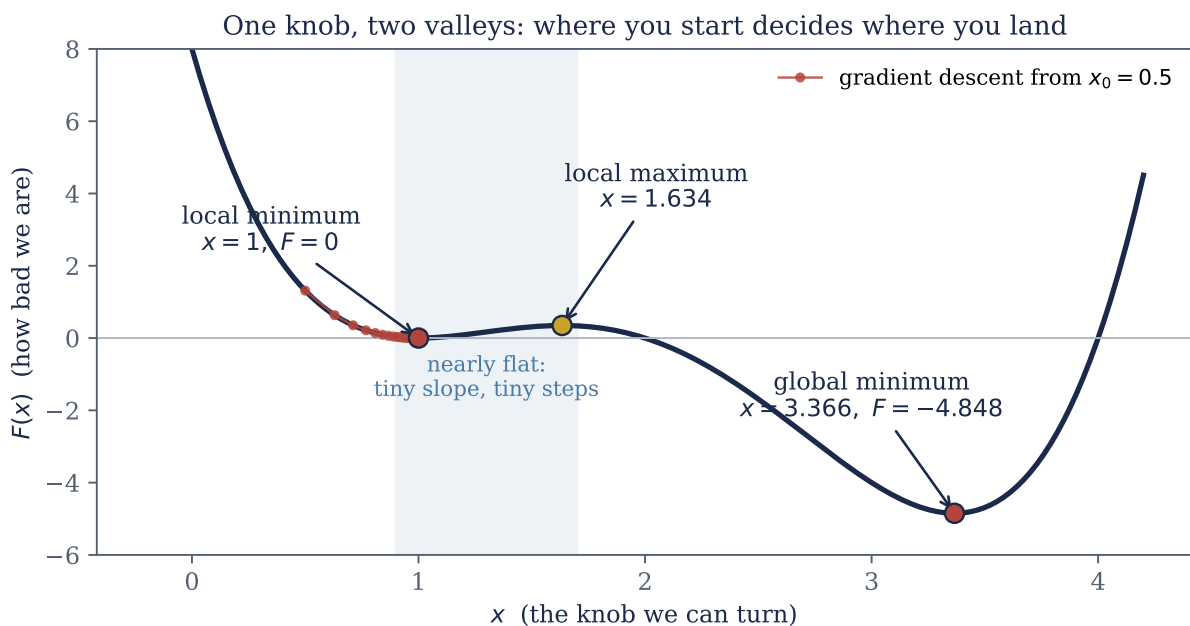
Step 6 — how deep is each valley?

$$F(1) = 0^2((1-3)^2 - 1) = 0$$

$$F(1.634) = (0.634)^2((-1.366)^2 - 1) = 0.4020 \times 0.8660 = 0.348$$

$$F(3.366) = (2.366)^2((0.366)^2 - 1) = 5.598 \times (-0.866) = -4.848$$

The valley at $x = 1$ has depth 0. The valley at $x = 3.366$ has depth -4.848 . The second is *far* better — and gradient descent has no way of knowing it exists.



Worked example 2: gradient descent walking into the wrong valley

Start at $x_0 = 0.5$ with $\eta = 0.02$, using $F'(x) = 4x^3 - 24x^2 + 42x - 22$ and the update $x \leftarrow x - \eta F'(x)$.

step	x	$F'(x)$	$F(x)$
0	0.5000	-6.500	1.3125
1	0.6300	-4.065	0.6321
2	0.7113	-2.829	0.3532
3	0.7679	-2.089	0.2146
4	0.8097	-1.604	0.1376
5	0.8418	-1.266	0.0916
6	0.8671	-1.019	0.0627
7	0.8874	-0.833	0.0439
8	0.9041	-0.689	0.0312
$\vdots$	$\vdots$	$\vdots$	$\vdots$
∞	1.0000	0	0

It converges beautifully. It reports success. It settles at $F = 0$ while a point at $F = -4.848$ existed the whole time, and it never had the slightest hint. *Any* start to the left of 1.634 ends here; only a start to the right of 1.634 finds the good valley.

Notice the second column too: the gradient shrinks every step. The closer you get, the slower you go. Descent naturally decelerates near a minimum — which is exactly why flat regions, where the gradient is already small *far* from any minimum, are so expensive.

Why high dimensions make this catastrophic

The slides give a lovely counting argument. Take a loss that splits into independent one-variable pieces:

$$F(x_1, x_2, \dots, x_d) = A_1(x_1) + A_2(x_2) + \dots + A_d(x_d).$$

The partial derivative with respect to x_i only sees A_i , because every other term is a constant as far as x_i is concerned:

$$\frac{\partial F}{\partial x_i} = A'_i(x_i).$$

So $\nabla F = \mathbf{0}$ exactly when *every* $A'_i(x_i^*) = 0$ simultaneously. If A_i has k_i local minima, you may pick any one of them independently in each coordinate, and the total count of local minima of F is the product

$$\prod_{i=1}^d k_i.$$

In plain words

It is a combination lock. Each dial can settle in one of k notches; the lock has k^d settings in total, and only one of them opens the door. With a modest $d = 10$ and $k = 3$ that is $3^{10} = 59,049$ local minima. With $d = 50$ it is about 7×10^{23} . Searching them is hopeless.

Flat regions

Look again at the shaded band in the figure, and at slide 15's picture. Over a long stretch the curve is almost horizontal, so $F'(x) \approx 0$, so the step $\eta F'(x) \approx 0$. You are not at a minimum; you are simply on a plateau, and you inch across it.

If the gradient on a plateau is 0.001 and $\eta = 0.01$, each step moves you 10^{-5} . To cross a plateau of width 2 you need about 200,000 steps. On a real dataset that is hours of GPU time spent going nowhere.

In plain words

A salt flat in fog. The ground is level in every direction, so your “walk downhill” rule gives you almost nothing to act on. You shuffle. The cure is *momentum*: remember the direction you were already moving and keep rolling, like a ball that carries speed across the flat instead of re-deciding at every footstep.

6. Differential curvature: the canyon problem

Two bowls

$$L_{\text{circle}} = x^2 + y^2, \quad L_{\text{ellipse}} = x^2 + 4y^2.$$

Both are convex, both have exactly one minimum at the origin. There is no local-minimum problem here at all — and yet one is easy and the other is not.

Second derivatives. For the circular bowl,

$$\frac{\partial^2 L}{\partial x^2} = 2, \quad \frac{\partial^2 L}{\partial y^2} = 2.$$

For the elliptical bowl,

$$\frac{\partial^2 L}{\partial x^2} = 2, \quad \frac{\partial^2 L}{\partial y^2} = 8.$$

Curvature is $4\times$ larger in y than in x . That single ratio is the whole story.

The circular bowl: one step, if you like

Gradient: $\nabla L = (2x, 2y)$. Update from a point (x, y) :

$$\begin{aligned} x &\leftarrow x - \eta(2x) = (1 - 2\eta)x \\ y &\leftarrow y - \eta(2y) = (1 - 2\eta)y \end{aligned}$$

Both coordinates shrink by the *same* factor $(1 - 2\eta)$. Choose $\eta = 0.5$:

$$(1 - 2\eta) = 0 \implies (x, y) \rightarrow (0, 0) \text{ in a single step, from anywhere.}$$

This is what slide 19 means by “a sufficiently large step-size can take us directly to the optimum in one step”. The gradient points straight at the target because the level sets are circles.

The elliptical bowl: no such luck

Gradient: $\nabla L = (2x, 8y)$. Update:

$$\begin{aligned} x &\leftarrow (1 - 2\eta)x \\ y &\leftarrow (1 - 8\eta)y \end{aligned}$$

Now the two coordinates shrink by *different* factors, and one η must serve both.

Stability requirement. For a coordinate not to blow up we need the factor’s magnitude below 1:

$$|1 - 2\eta| < 1 \implies \eta < 1, \quad |1 - 8\eta| < 1 \implies \eta < 0.25.$$

So η is capped at 0.25 **by the steep direction**, even though the shallow direction would happily accept a step four times larger. The most sensitive parameter puts a ceiling on everyone else’s learning rate.

What that costs you. Take $\eta = 0.24$, just inside the limit:

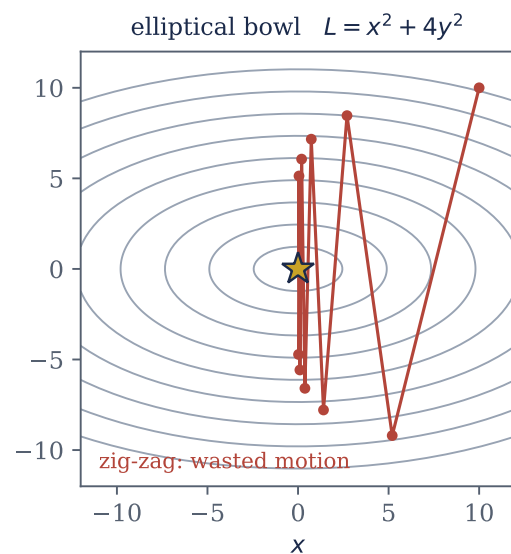
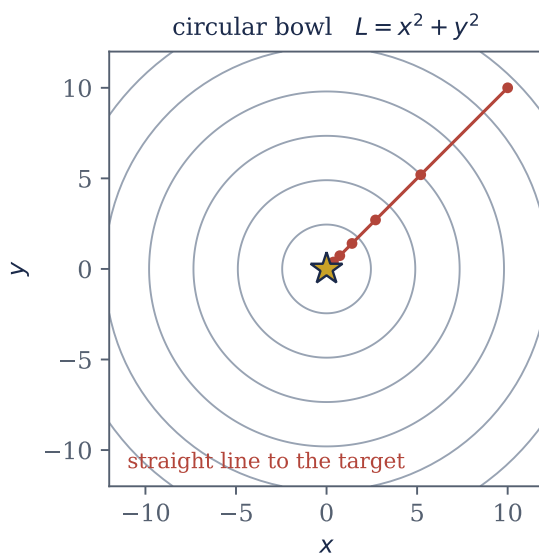
$$y\text{-factor} = 1 - 8(0.24) = -0.92, \quad x\text{-factor} = 1 - 2(0.24) = 0.52.$$

The negative y -factor is the zig-zag: every step overshoots the valley floor and lands on the opposite wall, with the magnitude shrinking only 8% each time.

step	x	y	L	
0	10.000	10.000	500.0	start
1	5.200	-9.200	365.6	y has flipped sign — overshoot
2	2.704	8.464	293.9	flipped back
3	1.406	-7.787	244.5	still bouncing
4	0.731	7.164	205.8	x is essentially solved
5	0.380	-6.591	173.9	y barely improving
6	0.198	6.064	147.1	

After 6 steps the x -coordinate has travelled from 10 to 0.2 — done. The y -coordinate has gone from 10 to 6.06 while ricocheting between positive and negative six times. Most of the movement was sideways, not downwards.

Same learning rate $\eta = 0.24$, same start (10, 10)



In plain words

Skiing down a narrow gully. Straight down the gully is the direction you actually want to go, but the steepest slope at your feet points at the *wall*. So you shoot across, hit the far wall, shoot back. You are always going “downhill” and you are barely making progress. This is why the slides say the steepest-descent direction is only the best *instantaneous* direction, not the best direction in the long run.

Why the gradient is perpendicular to the contour line

Slide 19 states it; here is the argument in full.

A contour line is the set of points where the function has one fixed value, say $F = c$. Take a tiny displacement $\delta \mathbf{x}$ that stays *on* that contour. By the first-order Taylor expansion,

$$F(\mathbf{x} + \delta \mathbf{x}) \approx F(\mathbf{x}) + \nabla F \cdot \delta \mathbf{x}.$$

But moving along a contour does not change F at all, so $F(\mathbf{x} + \delta\mathbf{x}) = F(\mathbf{x})$, hence

$$\nabla F \cdot \delta\mathbf{x} = 0.$$

A zero dot product means the two vectors are at right angles. Since $\delta\mathbf{x}$ points *along* the contour, ∇F points *across* it.

The gradient always crosses the contour lines at 90° . On circular contours “across the contour” happens to point at the centre. On stretched contours it does not — and that gap is exactly the zig-zag.

7. Revisiting normalisation: the fix, with the numbers

Now the two ideas — bad scaling and bad curvature — meet. This is slides 22–23, worked out end to end.

The toy data

Guns x_1 (per capita)	Butter x_2 (oz per capita)	Happiness y
0.1	25	7
0.8	10	1
0.4	10	4

Model: $\hat{y} = w_1x_1 + w_2x_2$. Loss:

$$J(\mathbf{w}) = (0.1w_1 + 25w_2 - 7)^2 + (0.8w_1 + 10w_2 - 1)^2 + (0.4w_1 + 10w_2 - 4)^2.$$

Measuring the damage

The second derivatives of J tell us the curvature in each direction.

$$\frac{\partial^2 J}{\partial w_1^2} = 2 \sum_i x_{i1}^2, \quad \frac{\partial^2 J}{\partial w_2^2} = 2 \sum_i x_{i2}^2, \quad \frac{\partial^2 J}{\partial w_1 \partial w_2} = 2 \sum_i x_{i1} x_{i2}.$$

Compute each sum:

$$\begin{aligned} \sum x_{i1}^2 &= 0.1^2 + 0.8^2 + 0.4^2 = 0.01 + 0.64 + 0.16 = 0.81 \\ \sum x_{i2}^2 &= 25^2 + 10^2 + 10^2 = 625 + 100 + 100 = 825 \\ \sum x_{i1} x_{i2} &= (0.1)(25) + (0.8)(10) + (0.4)(10) = 2.5 + 8 + 4 = 14.5 \end{aligned}$$

So the Hessian (the matrix of second derivatives) is

$$H = \begin{pmatrix} 1.62 & 29 \\ 29 & 1650 \end{pmatrix}.$$

The bowl is over a thousand times stiffer in w_2 than in w_1 . Its eigenvalues are $\lambda_{\min} = 1.11$ and $\lambda_{\max} = 1650.5$, so the **condition number** is

$$\kappa = \frac{\lambda_{\max}}{\lambda_{\min}} = \frac{1650.5}{1.11} \approx 1487.$$

Roughly speaking, κ is the “narrowness” of the canyon; the number of gradient-descent steps you need grows in proportion to it. This canyon is about $\sqrt{1487} \approx 39$ times longer than it is wide.

Standardising the columns, in full

Guns column {0.1, 0.8, 0.4}:

$$\mu_1 = \frac{0.1 + 0.8 + 0.4}{3} = \frac{1.3}{3} = 0.4333$$

deviations : -0.3333, +0.3667, -0.0333

$$\sigma_1^2 = \frac{(-0.3333)^2 + (0.3667)^2 + (-0.0333)^2}{3} = \frac{0.1111 + 0.1344 + 0.0011}{3} = 0.0822$$

$$\sigma_1 = \sqrt{0.0822} = 0.2867$$

Butter column {25, 10, 10}:

$$\mu_2 = \frac{25 + 10 + 10}{3} = \frac{45}{3} = 15$$

deviations : +10, -5, -5

$$\sigma_2^2 = \frac{100 + 25 + 25}{3} = \frac{150}{3} = 50$$

$$\sigma_2 = \sqrt{50} = 7.0711$$

Apply $z = (x - \mu)/\sigma$ to every entry:

x_1	x_2	z_1	z_2
0.1	25	$(0.1 - 0.4333)/0.2867 = -1.163$	$(25 - 15)/7.071 = 1.414$
0.8	10	$(0.8 - 0.4333)/0.2867 = 1.279$	$(10 - 15)/7.071 = -0.707$
0.4	10	$(0.4 - 0.4333)/0.2867 = -0.116$	$(10 - 15)/7.071 = -0.707$

Measuring the repair

Recompute the same three sums on the standardised columns:

$$\sum z_{i1}^2 = (-1.163)^2 + (1.279)^2 + (-0.116)^2 = 1.352 + 1.635 + 0.014 = 3.000$$

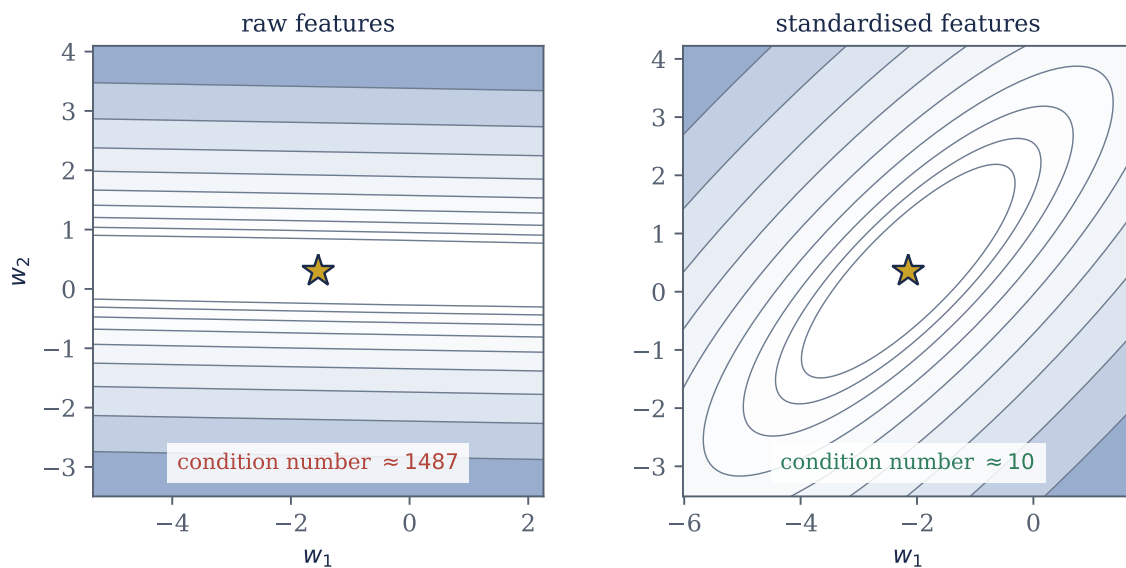
$$\sum z_{i2}^2 = (1.414)^2 + (-0.707)^2 + (-0.707)^2 = 2.000 + 0.500 + 0.500 = 3.000$$

$$\sum z_{i1}z_{i2} = (-1.163)(1.414) + (1.279)(-0.707) + (-0.116)(-0.707) = -2.466$$

The first two are exactly 3 — and that is not a coincidence. Dividing a column by its own standard deviation forces $\sum_i z_{ij}^2 = n$ for every column j , so *every diagonal entry of the Hessian becomes* $2n$. The new Hessian is

$$H_{\text{std}} = \begin{pmatrix} 6 & -4.93 \\ -4.93 & 6 \end{pmatrix}, \quad \lambda_{\min} = 1.07, \quad \lambda_{\max} = 10.93, \quad \kappa \approx 10.2.$$

The same problem, re-measured: a razor blade becomes a bowl



The condition number dropped from about 1487 to about 10 — a $145\times$ improvement — and we did not touch the model, the algorithm or the data's meaning. We only changed the units. That is the cheapest win available in all of machine learning.

Notice κ is 10, not 1. The leftover stretch comes from the off-diagonal term -4.93 : guns and butter are *correlated* in this dataset, so the two knobs are not independent, and the contours stay tilted (see the right panel above). Scaling equalises the axes; it does not decorrelate them. Removing that residual tilt is what PCA-whitening, and in spirit what Adam and second-order methods, are for.

8. The whole toolkit on one page

Symptom	What is going on	What to do about it
Loss becomes NaN in the first few steps	Overflow from huge initial weights or unscaled features	Initialise in $[-1, 1]$; scale the features; lower η
Training loss = 0, test loss terrible	Overfitting: more parameters than the data can pin down	More data, fewer parameters, regularisation, early stopping, cross-validation
Loss oscillates or explodes	η exceeds $2/\lambda_{\max}$ in the steepest direction	Reduce η ; standardise features; use an adaptive method
Loss falls fast, then crawls forever	Zig-zag in a badly conditioned canyon	Standardise; add momentum; use Adam/RMSProp
Loss is flat from the very start	Plateau, or a dead activation, or η far too small	Momentum; better initialisation; raise η ; check the gradient code
Different runs, different answers	Multiple local minima, different random starts	Multiple restarts and keep the best; accept it — in deep nets most minima are about equally good

In plain words

Almost every trick in modern optimisation is one of two ideas. **(1) Reshape the landscape** so plain gradient descent works — that is normalisation, batch-norm, better parameterisation. **(2) Give the walker better instincts** — momentum (keep rolling), adaptive learning rates (a private η per parameter, so the stiff knob is not held hostage by the hair-trigger one), and second-order methods (feel the curvature, not just the slope).

9. Check yourself

Q1. A model has 8 parameters and you train it on 6 rows. Should you be pleased when the training error hits exactly zero?

Q2. For $L = x^2 + 9y^2$, what is the largest learning rate that will not blow up? Which direction imposes the limit?

Q3. A column of your data has $\mu = 40$ and $\sigma = 8$. What is the standardised value of the entry 56?

Q4. You run gradient descent twice from two different random starts and get two different final losses. What does this tell you about the loss surface?

Q5. Why does the gradient meet a contour line at a right angle?

A1. No. With 8 unknowns and 6 equations the system is underdetermined, so infinitely many weight vectors give zero error; the optimiser hands you one arbitrarily and it is probably a memoriser. Judge on held-out data.

A2. $\nabla L = (2x, 18y)$, so the update factors are $(1 - 2\eta)$ and $(1 - 18\eta)$. Stability needs $|1 - 18\eta| < 1$, i.e. $\eta < 2/18 = 1/9 \approx 0.111$. The y -direction — the one with the larger curvature $\partial^2 L / \partial y^2 = 18$ — sets the ceiling.

A3. $z = (56 - 40)/8 = 2$. The entry sits two standard deviations above the mean.

A4. The surface is non-convex: it has more than one local minimum, and each start fell into a different basin. (A convex bowl would return the same answer every time.)

A5. Along a contour F does not change, so $\nabla F \cdot \delta \mathbf{x} = 0$ for any small step $\delta \mathbf{x}$ taken along it; a zero dot product means perpendicular.

Glossary in ordinary words

Term	Plain meaning
Objective / loss $J(\mathbf{w})$	The bill for being wrong. Training = making the bill small.
Gradient ∇J	The direction of steepest <i>increase</i> . We walk the other way.
Learning rate η	Stride length. Too small: forever. Too large: you bounce out of the valley.
Local minimum	Bottom of <i>a</i> valley. Global minimum: bottom of the <i>deepest</i> valley.
Flat region / plateau	Level ground far from any minimum. Gradient ≈ 0 , so progress ≈ 0 .
Curvature (F'' , Hessian)	How fast the slope itself changes. High curvature = a tight, steep wall.
Condition number κ	Stiffest direction $\div$ softest direction. Big κ = narrow canyon = slow training.
Overfitting	Scoring 100% by memorising the answer key instead of learning the subject.
Standardisation	Rewriting every column in units of “standard deviations from its own mean”.

Companion notes for ZC416 Lecture 10 • read alongside the official slides